Práctico 12 - Recursión

Programación Imperativa
InCo - Facultad de Ingenieía, Udelar

- Los subprogramas pedidos deben implementarse usando recursión. *No debe* usar `for`, `while` ni `repeat`.
- Puede suponer que los parámetros satisfacen las precondiciones indicadas en cada ejercicio.

1. Considere los siguientes subprogramas recursivos.

   (a)
   ```
   {PRE: n >= 1 }
   function f(n : integer) : integer;
   begin
      if n = 1 then
         f := 0
      else
         f := 1 + f(n div 2)
   end;
   ```

   Escriba la secuencia de llamadas recursivas que se realizan e indique el valor retornado por la función, cuando se invoca `f(13)` y `f(8)`.

   (b)
   ```
   {PRE: n >= 0 }
   function g(n : integer) : integer;
   begin
      if n < 10 then
         g := n
      else
         g := g(n div 10) + (n mod 10)
   end;
   ```

   Escriba la secuencia de llamadas recursivas que se realizan e indique el valor retornado por la función, cuando se invoca `g(731)` y `g(2048)`.

   (c)
   ```
   {PRE: n >= 0, m >= 0 }
   function h(n, m : integer) : integer;
   begin
      if n = m then h := n
      else if n < m then h := h (n, m-n)
      else h := h (n-m, m)
   end;
   ```

   Escriba la secuencia de llamadas recursivas que se realizan e indique el valor retornado por la función, cuando se invoca `h(11,27)` y `h(8,128)`.

2. Se quiere trabajar con la representación decimal de los enteros no negativos; esta representación comienza con cero solamente cuando queremos representar el entero cero que tiene un único dígito. Implemente los siguientes subprogramas.

   (a) La función que devuelve la cantidad de dígitos de su argumento.

   ```
   {PRE: n >= 0 }
   function cantDigitos(n : integer) : integer;
   ```

   (b) La función que devuelve la suma de los dígitos de su argumento.

   ```
   {PRE: n >= 0 }
   function sumaDigitos(n : integer) : integer;
   ```

(c) La función que devuelve `true` si el dígito `d` aparece en la representación decimal de `n`, y `false` en caso contrario.

```
{PRE: n >= 0, 0 <= d <= 9 }
function contieneDigito(n, d : integer) : boolean;
```

3. (a) Implemente recursivamente la función que calcula la potencia b^n de base b:

```
{PRE: b >= 0, n >= 0 }
function potencia (b, n : integer): integer;
```

(b) Implemente recursivamente la función que calcula el logaritmo entero $\lfloor \log_b n \rfloor$ de base b:

```
{PRE: b > 1, n > 0 }
function logaritmo (b, n : integer): integer;
```

4. Implemente recursivamente el procedimiento

```
{PRE: dividendo >= 0, divisor > 0 }
procedure divisionPorResta(dividendo, divisor : integer;
                           var cociente, resto : integer);
```

que calcula el cociente y el resto de la división entera de `dividendo` entre `divisor`, usando únicamente restas, comparaciones y sumas.

5. **Rayuela.** Una hilera de baldosas está numerada desde 0 en adelante. Al comenzar, un niño se encuentra en la baldosa 0. Desde una baldosa `i` puede realizar uno de los siguientes movimientos:

- saltar a la baldosa `i+1`;
- saltar a la baldosa `i+2`.

Implemente una función recursiva

```
{PRE: n >= 0 }
function caminos(n : integer) : integer;
```

que devuelva la cantidad de caminos diferentes que permiten llegar exactamente a la baldosa `n`. Considere que existe un único camino para llegar a la baldosa 0.

Por ejemplo, para `n=3` existen tres caminos:

$$(0,1,2,3), \qquad (0,2,3), \qquad (0,1,3).$$

6. Una misma función puede definirse de múltiples maneras. En este ejercicio vemos definiciones del número combinatorio $\binom{n}{k}$, la cantidad de subconjuntos de k elementos que pueden construírse en un universo de n elementos.

(a) Implemente la función recursiva

```
{PRE: m >= n >= 0 }
function combinaciones(m, n : integer) : integer;
```

que calcula $\binom{m}{n}$, suponiendo $m \geq n \geq 0$, a partir de las siguientes ecuaciones:

$$\binom{m}{0} = 1, \qquad \binom{m}{m} = 1,$$

$$\binom{m}{n} = \binom{m-1}{n} + \binom{m-1}{n-1} \quad \text{si } m > n > 0.$$

Pruebe su función calculando $\binom{5}{2}$, $\binom{6}{3}$, $\binom{10}{0}$, $\binom{24}{12}$.

(b) Considere la ecuación

$$k\binom{n}{k} = n\binom{n-1}{k-1} \text{ si } n \geq k \geq 1.$$

A partir de ella, implemente la función recursiva

```
{PRE: n >= k >= 0 }
function combinaciones2(n, k : integer) : integer;
```

que calcula $\binom{n}{k}$, suponiendo $n \geq k \geq 0$. Pruebe su función calculando $\binom{5}{2}$, $\binom{6}{3}$, $\binom{10}{0}$, $\binom{24}{12}$.

7. Considere las definiciones:

```
const MAX = ...;
type TArreglo = array [1..MAX] of integer;
```

Implemente los siguientes subprogramas:

(a) `function estaOrdenado(A : TArreglo) : boolean;`

que retorna `true` si los elementos de `A` están ordenados de menor a mayor y `false` en caso contrario. Siga el esquema

```
function estaOrdenado(A : TArreglo) : boolean;
   function rec (L: integer): boolean;
   begin
      ...
   end;
begin estaOrdenado := rec (1) end;
```

(b) `function ultimaPos(A : TArreglo; v : integer) : integer;`

que retorna la última posición de `A` que contiene el valor `v`. Si `v` no está en el arreglo, debe devolver 0. Siga el esquema

```
function ultimaPos (A : TArreglo; v: integer): integer;
   function rec ( R : integer ): integer ;
   begin
   ...
   end ;
begin ultimaPos := rec ( MAX ) end ;
```

8. Considere las definiciones:

```
const MAX = ...;
type palabra = record
       info: array[1..MAX] of char;
       tope: 0..MAX;
     end;
```

Como siempre, en el arreglo con tope la información relevante no comprende necesariamente a todo el arreglo. Implemente los siguientes subprogramas:

(a) `function iguales(pal1, pal2 : palabra) : boolean;`

que devuelve `True` si y solamente si ambas palabras son iguales. Siga el esquema

```
function iguales (pal1, pal2 : palabra): boolean;
   function rec ( L : integer ): boolean ;
   begin
   ...
   end ;
begin iguales := (pal1.tope = pal2.tope) and rec (1) end ;
```

(b) `function reversas(pal1, pal2 : palabra) : boolean;`

que devuelve `True` si y solamente si una de las palabras leída de izquierda a derecha es igual a la otra leída de derecha a izquierda. Siga el esquema

```
function reversas (pal1, pal2 : palabra): boolean;
   function rec ( L, R : integer ): boolean ;
   begin
   ...
   end ;
begin reversas := (pal1.tope = pal2.tope) and rec (1, pal1.tope) end ;
```

(c) `function prefija(pal1, pal2 : palabra) : boolean;`

que devuelve `True` si y solamente si la primera palabra es prefija de la segunda. Siga el esquema

```
function prefija (pal1, pal2 : palabra): boolean;
   function rec ( L : integer ): boolean ;
   begin
   ...
   end ;
begin prefija := (pal1.tope <= pal2.tope) and rec (1) end ;
```

(d) `function subcadena(pal1, pal2 : palabra) : boolean;`

que devuelve `True` si y solamente si la primera palabra es subcadena de la segunda. Siga el esquema

```
function subcadena (pal1, pal2 : palabra): boolean;
   function rec ( L1, L2 : integer ): boolean ;
   begin
   ...
   end ;
begin subcadena := (pal1.tope <= pal2.tope) and rec (1, 1) end ;
```

(e) `function palindrome(pal : palabra) : boolean;`

que devuelve `True` si y solamente si la palabra es palíndrome. Siga el esquema

```
function palindrome (pal : palabra): boolean;
   function rec ( L, R : integer ): boolean ;
   begin
   ...
   end ;
begin palindrome := rec (1, pal.tope) end ;
```

(f) `procedure invertir(var pal : palabra);`

que invierte la palabra dada. Siga el esquema

```
procedure invertir (var pal : palabra);
   procedure rec ( L, R : integer );
   begin
   ...
   end ;
begin rec (1, pal.tope) end ;
```

Ejercicios adicionales

Estos ejercicios señalan aspectos interesantes o problemáticos de la recursión que quedan por fuera del curso; deben entenderse como ejercicios adicionales para el lector interesado.

1. Muchas veces es sencillo demostrar que los códigos recursivos son correctos usando inducción completa. En este ejercicio estudiamos las implementaciones del ejercicio 1.

   (a) Demuestre que el resultado de invocar `g` con el argumento n positivo es la suma de sus dígitos; use inducción completa sobre la cantidad de dígitos de n.

(b) Demuestre que el resultado de invocar `f` con el valor 2^k natural es k; use inducción completa sobre k. Demuestre que al invocar `f` con un valor n entero positivo, el resultado es $\lfloor \log_2 n \rfloor$.

2. Cuando una función realiza múltiples invocaciones recursivas puede suceder que computen un mismo subproblema múltiples veces. Por ejemplo, si despliego los cómputos necesarios para calcular Fibonacci, vemos que ...

$$\mathit{fib}(n+3) = \mathit{fib}(n+2) + \mathit{fib}(n+1) = \mathbf{fib}(\mathbf{n+1}) + \mathit{fib}(n) + \mathbf{fib}(\mathbf{n+1}) = \ldots$$

el cómputo de $\mathit{fib}(n+1)$ se realiza dos veces.

El ejercicio 6 propone computar una función de dos formas diferentes; en una aparece este problema, en la otra no. Compute los números $\binom{24}{12}, \binom{26}{13}, \binom{28}{14}, \binom{30}{15}, \ldots$ y observe los tiempos de ejecución.